CENTRAL BOARD OF SECONDARY EDUCATION

JAWAHAR NAVODAYA VIDYALAYA

TARIKHET ALMORA UTTARAKHAND 263663

COMPUTER SCIENCE (083)

PROJECT FILE

On

QUIZ SYSTEM USING TEXT FILE HANDLING

SUBMITTED BY :

S. No	Name	CBSE Roll
1.		
2.		
3.		
4.		

CLASS : XII
SECTION : SCIENCE

INTERNAL EXAMINER (PANKAJ BHILINGWAL)	EXTERNAL EXAMINER	PRINCIPAL (D.S RAWAT)

ACKNOWLEDGEMENT

*I wish to express my deep sense of gratitude and indebtedness to our learned teacher **Pankaj Bhilingwal**, PGT Computer Science, J.N.V Tarikhet Almora for his invaluable help, advice and guidance in the preparation of this project file.*

*I am also greatly indebted to our principal **D.S Rawat** and school authorities for providing me with the facilities and requisite laboratory conditions for making this project file.*

I also extend my thanks to a number of teachers, my classmates and friends who helped me to complete this project file successfully.

NAME :

S. No	Name	CBSE Roll
1.		
2.		
3.		
4.		

CLASS : XII
SECTION : SCIENCE

CERTIFICATE

This is to certify that,

S. No	Name	CBSE Roll
1.		
2.		
3.		
4.		

Student of Class XII, J.N.V Tarikhet Almora has completed the PROJECT FILE during the academic year 2026-27 towards partial fulfilment of credit for the Computer Science Project evaluation of CBSE and submitted satisfactory report, as compiled in the following pages, under my supervision.

Internal Examiner
Signature

External Examiner
Signature

PREFACE

*This project report outlines the development of a **Quiz Management System** using Python. The project simulates a quiz system where students can register, log in, and participate in quizzes. Administrators can add quiz questions, view users, and monitor quiz results. The system uses file handling to store data persistently.*

I also feel that it is the right time to thank my teacher, who guided me during my project. I also feel that it is the right time to thank my parents, friends and my project partners for their constant assistance, cooperation extended by them time to time.

Table of Contents

About Project 5
Introduction 5
Modules Used 5
1. time Module: 5
2. random Module: 5
File Handling 5
Functions Overview 5
1. User Registration Functions: 5
2. User Login Functions: 5
3. Admin Functions: 6
4. Quiz Functions: 6
Core Features Explained 6
1. User Registration: 6
2. Admin Capabilities: 6
3. Taking the Quiz: 6
4. Storing Results: 7
Menu Structure 7
1. Main Menu: 7
2. Admin Menu: 7
3. User Menu: 7
Error Handling and Input Validation 7
Conclusion 8
Flow Chart 8
Project Code 9
Output 17
Technological requirements 23
Software 23
Hardware 23
References 24

About Project

Introduction

The **Quiz Management System** is a Python-based application that allows users to register, log in, and participate in quizzes. It supports two types of users: administrators and regular users. Administrators have special permissions to add questions and view results, while users can take quizzes and see their performance. The program uses text files for data storage, handling user registration, quiz questions, and results.

Modules Used

1. **time Module:** This is used to track the time taken by users during the quiz. The time.time() function is used to record the start and end time of the quiz, helping calculate the total time spent on the quiz.
2. **random Module:** The random.shuffle() function is used to randomize the order of quiz questions to ensure that every quiz attempt is unique.

File Handling

The program uses three text files to store persistent data:

- users.txt: Stores registered user details (username, password, class, section, etc.).
- questions.txt: Stores quiz questions, options, and correct answers.
- results.txt: Stores users' quiz performance, including scores, correct and incorrect answers, and time taken.

Functions Overview

1. User Registration Functions:

- **generate_username(cls, section, roll)**: This function creates a unique username for each student based on their class, section, and roll number (e.g., "XI-SC-001").
- **save_user(username, password, name, roll, cls, section, dob, role)**: Appends the user's details to the users.txt file for persistence.
- **register()**: Handles the entire user registration process, asking for user details and saving them to the file. This ensures that the user's information is saved and can be retrieved later during login.

2. User Login Functions:

- **load_users()**: Reads the users.txt file and returns a dictionary containing all registered users.
- **login()**: Prompts the user for username and password, verifies them against the stored user data, and returns the role of the user (either "admin" or "user"). If the credentials are invalid, it prompts an error.

3. Admin Functions:

- **add_question()**: Prompts the admin to add a quiz question along with four options, the correct answer, and a category. The question is saved to the questions.txt file.
- **view_questions()**: Allows the admin to view all the quiz questions stored in the questions.txt file.
- **view_registered_users()**: Displays a list of all registered users along with their details, which helps the admin monitor participants.
- **leaderboard()**: Displays the top 10 quiz performers by sorting the quiz results based on their scores.
- **view_all_results()**: Shows all users' quiz results, allowing the admin to review participation and performance.
- **search_user_result()**: Enables the admin to search for the quiz result of a particular user by their username.

4. Quiz Functions:

- **load_questions()**: Reads the quiz questions from the questions.txt file and returns them in a structured format (with question text, options, correct answers, and categories).
- **play_quiz(username)**: This function runs the actual quiz. It presents questions to the user, records their answers, and calculates the final score. Time taken and number of correct/incorrect answers are saved to the results.txt file.
- **save_result(username, score, correct, incorrect, time_taken)**: Saves the quiz result data (score, correct answers, incorrect answers, and time taken) to the results.txt file for later retrieval.

Core Features Explained

1. **User Registration:** The registration process gathers the student's name, roll number, class (XI or XII), section (Science or Humanities), and date of birth, and then prompts the user to create a password. The system generates a unique username by combining class, section, and roll number. After registration, this data is stored in a text file, ensuring that students can log in later to take quizzes.
2. **Admin Capabilities:** The admin has special permissions to manage the quiz content. They can add quiz questions, view the leaderboard, and examine individual results. This role allows the admin to update the quiz system as needed, keeping the question bank fresh and reviewing user performance.
3. **Taking the Quiz:** The quiz is a timed event where the user is presented with randomly shuffled questions. The system tracks how long the user takes to complete the quiz, deducts points for incorrect answers (0.25 points), and gives a score based on the number of correct responses. This gamified aspect of the quiz is designed to engage students while testing their knowledge.

4. **Storing Results:** Each quiz attempt is stored in a results file, including the user's score, number of correct and incorrect answers, and the time taken to complete the quiz. This data can be used to track user performance over time, providing insights into their progress.

Menu Structure

1. Main Menu:

1. Register
2. Login
3. Exit

2. Admin Menu:

4. Add Question
5. View Questions
6. View Registered Users
7. View Leaderboard
8. View All Results
9. Search User Result
10. Logout

3. User Menu:

11. Play Quiz
12. Logout

This structure ensures that both admins and users have clearly defined actions they can take after logging in.

Error Handling and Input Validation

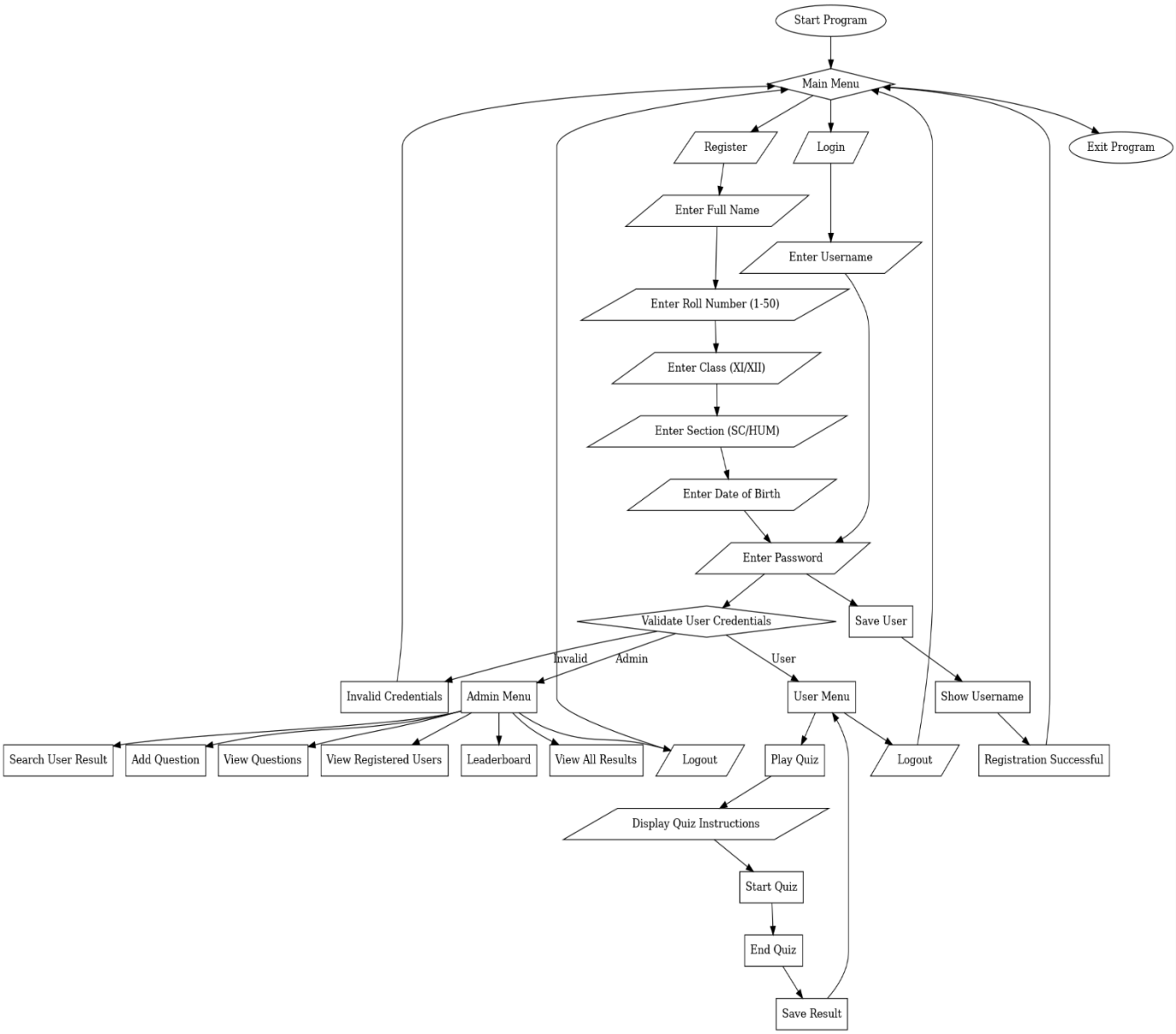
Throughout the code, validation mechanisms ensure that only valid inputs are accepted:

- During registration, roll numbers are checked to be between 1 and 50, and class/section inputs are validated to ensure they are either XI or XII, and SC or HUM.
- Password validation is done to check whether the entered username and password match during login.
- Admins can only add valid questions with one correct option (A/B/C/D), and they cannot enter duplicate questions.

Conclusion

This project demonstrates the power of Python for building an interactive quiz system with both user and admin functionalities. The use of file handling allows the system to maintain persistent data, and the randomization of questions makes each quiz attempt unique. The system is scalable, allowing the admin to add new quiz questions and more users to register, making it a flexible and useful tool for educational purposes.

Flow Chart



Project Code

```
import time
import random

# File paths
users_file = 'users.txt'
questions_file = 'questions.txt'
results_file = 'results.txt'

# Helper functions
def load_users():
    users = {}
    try:
        with open(users_file, 'r') as f:
            for line in f:
                username, password, name, roll, cls, section, dob, role = line.strip().split(',')
                users[username] = {'password': password, 'name': name, 'roll': roll, 'class': cls, 'section': section, 'dob': dob, 'role': role}
    except FileNotFoundError:
        pass
    return users

def save_user(username, password, name, roll, cls, section, dob, role):
    with open(users_file, 'a') as f:
        f.write(f"{username},{password},{name},{roll},{cls},{section},{dob},{role}\n")

def generate_username(cls, section, roll):
    return f"{cls}{section}{str(roll).zfill(3)}"

def register():
    name = input("Enter your full name: ")
    roll = int(input("Enter roll number (1-50): "))
    while not (1 <= roll <= 50):
```

```
        print("Invalid roll number. Please enter a number between 1 and 50.")
        roll = int(input("Enter roll number (1-50): "))

    cls = input("Enter class (XI/XII): ").upper()
    while cls not in ["XI", "XII"]:
        print("Invalid class. Please enter XI or XII.")
        cls = input("Enter class (XI/XII): ").upper()

    section = input("Enter section (SC/HUM): ").upper()
    while section not in ["SC", "HUM"]:
        print("Invalid section. Please enter SC or HUM.")
        section = input("Enter section (SC/HUM): ").upper()

    dob = input("Enter date of birth (DD/MM/YYYY): ")
    password = input("Enter a password: ")

    username = generate_username(cls, section, roll)
    save_user(username, password, name, roll, cls, section, dob, 'user')
    print(f"Registration successful! Your username is {username}")

def login():
    username = input("Enter username: ")
    password = input("Enter password: ")
    users = load_users()
    if username in users and users[username]['password'] == password:
        return users[username]['role'], username
    else:
        print("Invalid credentials!")
        return None, None

# Admin Functions
def add_question():
    question = input("Enter question: ")
    options = [input(f"Option {chr(65+i)}: ") for i in range(4)]
```

```
    correct = input("Enter correct option (A/B/C/D): ").upper()
    category = input("Enter category: ")
    save_question(question, options, correct, category)
    print("Question added successfully!")

def save_question(question, options, correct, category):
    with open(questions_file, 'a') as f:
        f.write(f"{question}|{'|'.join(options)}|{correct}|{category}\n")

def load_questions():
    questions = []
    try:
        with open(questions_file, 'r') as f:
            for line in f:
                parts = line.strip().split('|')
                if len(parts) == 7:
                    question_text = parts[0]
                    options = parts[1:5]
                    correct_answer = parts[5].upper()
                    category = parts[6]
                    questions.append({'question': question_text, 'options': options, 'correct': correct_answer, 'category':
category})
    except FileNotFoundError:
        print("Questions file not found.")
    return questions

def view_questions():
    questions = load_questions()
    for i, q in enumerate(questions):
        print(f"Q{i+1}: {q['question']}")
        for j, opt in enumerate(q['options']):
            print(f" {chr(65+j)}. {opt}")
        print(f"Correct: {q['correct']}, Category: {q['category']}\n")
```

```
def view_registered_users():
    users = load_users()
    print("\nRegistered Users:")
    for username, details in users.items():
        print(f"Username: {username}, Name: {details['name']}, Roll: {details['roll']}, Class: {details['class']}, Section: {details['section']}, DOB: {details['dob']}")


def leaderboard():
    results = load_results()
    sorted_results = sorted(results, key=lambda x: x['score'], reverse=True)
    print("\nLeaderboard:")
    for result in sorted_results[:10]:  # Show top 10 performers
        print(f"Username: {result['username']}, Score: {result['score']}")


def load_results():
    results = []
    try:
        with open(results_file, 'r') as f:
            for line in f:
                username, score, correct, incorrect, time_taken = line.strip().split(',')
                results.append({'username': username, 'score': float(score), 'correct': int(correct), 'incorrect': int(incorrect), 'time_taken': int(time_taken)})
    except FileNotFoundError:
        pass
    return results


def save_result(username, score, correct, incorrect, time_taken):
    with open(results_file, 'a') as f:
        f.write(f"{username},{score},{correct},{incorrect},{time_taken}\n")


def view_all_results():
    results = load_results()
    print("\nAll Users' Quiz Participation:")
    for result in results:
```

```
        print(f"Username: {result['username']}, Score: {result['score']}, Correct: {result['correct']}, Incorrect: {result['incorrect']}, Time Taken: {result['time_taken']} seconds")


def search_user_result():
    username = input("Enter the username to search: ")
    results = load_results()
    user_found = False
    for result in results:
        if result['username'] == username:
            print(f"Username: {result['username']}, Score: {result['score']}, Correct: {result['correct']}, Incorrect: {result['incorrect']}, Time Taken: {result['time_taken']} seconds")
            user_found = True
    if not user_found:
        print("User result not found.")


def admin_menu():
    while True:
        print("\nAdmin Menu:")
        print("1. Add Question")
        print("2. View Questions")
        print("3. View Registered Users")
        print("4. Leaderboard")
        print("5. View All Users' Results")
        print("6. Search User Result")
        print("7. Logout")
        choice = input("Enter your choice: ")
        if choice == '1':
            add_question()
        elif choice == '2':
            view_questions()
        elif choice == '3':
            view_registered_users()
        elif choice == '4':
            leaderboard()
        elif choice == '5':
```

```
            view_all_results()
        elif choice == '6':
            search_user_result()
        elif choice == '7':
            break
        else:
            print("Invalid choice!")

# User Functions
def display_instructions():
    print("\nInstructions:")
    print("1. Each correct answer gives you 1 point.")
    print("2. Each incorrect answer will deduct 0.25 points.")
    print("3. You have limited time to complete the quiz.")
    print("4. Answer carefully!\n")

def play_quiz(username):
    questions = load_questions()
    if not questions:
        print("No questions available!")
        return

    random.shuffle(questions)  # Shuffle questions for random order
    total_questions = len(questions)
    correct_answers = 0
    incorrect_answers = 0
    start_time = time.time()
    time_limit = total_questions * 20  # 20 seconds per question

    display_instructions()

    for i, q in enumerate(questions):
        remaining_time = time_limit - (time.time() - start_time)
        if remaining_time <= 0:
```

```
            print("\nTime is up!")
            break
        print(f"\nQ{i+1}: {q['question']}")
        for j, opt in enumerate(q['options']):
            print(f" {chr(65+j)}. {opt}")
        print(f"Time remaining: {int(remaining_time)} seconds")
        answer = input("Your answer (A/B/C/D): ").upper()

        if answer == q['correct']:
            correct_answers += 1
        else:
            incorrect_answers += 1

    total_score = correct_answers * 1 - incorrect_answers * 0.25
    time_taken = int(time.time() - start_time)
    print(f"\nQuiz over! {username}'s score: {total_score}")
    print(f"Correct: {correct_answers}, Incorrect: {incorrect_answers}, Time taken: {time_taken} seconds")

    save_result(username, total_score, correct_answers, incorrect_answers, time_taken)

def user_menu(username):
    while True:
        print("\nUser Menu:")
        print("1. Play Quiz")
        print("2. Logout")
        choice = input("Enter your choice: ")
        if choice == '1':
            play_quiz(username)
        elif choice == '2':
            break
        else:
            print("Invalid choice!")

# Main program
```

```
def main():
    print("Welcome to the Quiz Game!")
    while True:
        print("\n1. Register")
        print("2. Login")
        print("3. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            register()
        elif choice == '2':
            role, username = login()
            if role == 'admin':
                admin_menu()
            elif role == 'user':
                user_menu(username)
        elif choice == '3':
            print("Thank you for using the Quiz Game!")
            break
        else:
            print("Invalid choice!")

if __name__ == "__main__":
    main()
```

Output

```
Welcome to the Quiz Game!

1. Register
2. Login
3. Exit
Enter your choice: 1
Enter your full name: Rohan Singh
Enter roll number (1-50): 2
Enter class (XI/XII): XII
Enter section (SC/HUM): SC
Enter date of birth (DD/MM/YYYY): 10/01/2008
Enter a password:   qwer
Registration successful! Your username is XIISC002

1. Register
2. Login
3. Exit
Enter your choice: 2
Enter username: XIISC002
Enter password: qwer

User Menu:
1. Play Quiz
2. Logout
Enter your choice: 1

Instructions:
1. Each correct answer gives you 1 point.
2. Each incorrect answer will deduct 0.25 points.
3. You have limited time to complete the quiz.
4. Answer carefully!
```

```
Q1: Processing unit of computer is :
 A. Mouse
 B. Keyboard
 C. Monitor
 D. CPU
Time remaining: 99 seconds
Your answer (A/B/C/D): A

Q2: Who wrote 'Romeo and Juliet'?
 A. Charles Dickens
 B. J.K. Rowling
 C. William Shakespeare
 D. Leo Tolstoy
Time remaining: 92 seconds
Your answer (A/B/C/D): C

Q3: Which planet is known as the Red Planet?
 A. Venus
 B. Mars
 C. Jupiter
 D. Saturn
Time remaining: 91 seconds
Your answer (A/B/C/D): B

Q4: What is 5 + 3?
 A. 6
 B. 7
 C. 8
 D. 9
Time remaining: 90 seconds
Your answer (A/B/C/D): D
```

```
Q5: What is the capital of France?
 A. Paris
 B. London
 C. Berlin
 D. Madrid
Time remaining: 89 seconds
Your answer (A/B/C/D): A

Quiz over! XIISC002's score: 2.5
Correct: 3, Incorrect: 2, Time taken: 10 seconds

User Menu:
1. Play Quiz
2. Logout
Enter your choice: 2


1. Register
2. Login
3. Exit
Enter your choice: 2
Enter username: admin
Enter password: admin123

Admin Menu:
1. Add Question
2. View Questions
3. View Registered Users
4. Leaderboard
5. View All Users' Results
6. Search User Result
7. Logout
Enter your choice: 1
```

```
Enter your choice: 1
Enter question: Example of Input Device is :
Option A: Monitor
Option B: Touch Screen
Option C: Printer
Option D: Speaker
Enter correct option (A/B/C/D): B
Enter category: Computer Basic
Question added successfully!
```

```
Admin Menu:
1. Add Question
2. View Questions
3. View Registered Users
4. Leaderboard
5. View All Users' Results
6. Search User Result
7. Logout
Enter your choice: 2
Q1: What is the capital of France?
 A. Paris
 B. London
 C. Berlin
 D. Madrid
Correct: A, Category: Geography

Q2: Which planet is known as the Red Planet?
 A. Venus
 B. Mars
```

```
Admin Menu:
1. Add Question
2. View Questions
3. View Registered Users
4. Leaderboard
5. View All Users' Results
6. Search User Result
7. Logout
Enter your choice: 3

Registered Users:
Username: admin, Name: Pankaj Bhilingwal, Roll: 00, Class: PGT, Sec
tion: COM-SC, DOB: 10/07/1995
Username: XISC014, Name: Pankaj, Roll: 14, Class: XI, Section: SC,
DOB: 10/07/2006
Username: XIISC002, Name: Rohan Singh, Roll: 2, Class: XII, Sectior
: SC, DOB: 10/01/2008
```

```
Admin Menu:
1. Add Question
2. View Questions
3. View Registered Users
4. Leaderboard
5. View All Users' Results
6. Search User Result
7. Logout
Enter your choice: 4

Leaderboard:
Username: XISC014, Score: 3.75
Username: XISC014, Score: 2.5
Username: XIISC002, Score: 2.5
Username: john, Score: 1.25
```

```
1. Add Question
2. View Questions
3. View Registered Users
4. Leaderboard
5. View All Users' Results
6. Search User Result
7. Logout
Enter your choice: 5

All Users' Quiz Participation:
Username: john, Score: 1.25, Correct: 2, Incorrect: 3, Time Taken:
8 seconds
Username: XISC014, Score: 3.75, Correct: 4, Incorrect: 1, Time Take
n: 11 seconds
Username: XISC014, Score: 2.5, Correct: 3, Incorrect: 2, Time Taken
: 17 seconds
Username: XIISC002, Score: 2.5, Correct: 3, Incorrect: 2, Time Take
n: 10 seconds
1. Add Question
2. View Questions
3. View Registered Users
4. Leaderboard
5. View All Users' Results
6. Search User Result
7. Logout
Enter your choice: 6
Enter the username to search: XIISC002
Username: XIISC002, Score: 2.5, Correct: 3, Incorrect: 2, Time Take
n: 10 seconds
```

```
Admin Menu:
1. Add Question
2. View Questions
3. View Registered Users
4. Leaderboard
5. View All Users' Results
6. Search User Result
7. Logout
Enter your choice: 7

1. Register
2. Login
3. Exit
Enter your choice: 3
Thank you for using the Quiz Game!
```

Technological requirements

Software

1. Python
2. VS Code

Hardware

1. Laptop/PC

References

1. **Python Official Documentation**
   Comprehensive resource for Python programming.
   https://docs.python.org/3/
2. **File Handling in Python**
   Tutorial on how to read from and write to files in Python, essential for managing user, question, and result data.
   https://realpython.com/read-write-files-python/
3. **Time Module in Python**
   Official Python documentation for the time module, used to track quiz time.
   https://docs.python.org/3/library/time.html
4. **Random Module in Python**
   Explanation of the random module, particularly the shuffle() function used to randomize quiz questions.
   https://docs.python.org/3/library/random.html
5. **Python Input Validation**
   Techniques for validating user input, critical for checking correct formats and valid data during registration and quiz.
   https://www.geeksforgeeks.org/python-input-validation/
6. **Building a Quiz System in Python**
   A helpful guide on how to create quiz systems and integrate file handling and scoring.
   https://www.tutorialspoint.com/python/python_online_quiz.htm
7. **Text File Parsing in Python**
   Example of how to split lines and parse data from text files, as used to handle questions and user information.
   https://www.programiz.com/python-programming/reading-csv-files
8. **Error Handling in Python**
   Tutorial on exception handling in Python, useful for managing invalid input or missing files.
   https://docs.python.org/3/tutorial/errors.html